

16. Chain of Responsibility Design Pattern in Java

16.1 Introduction

The **Chain of Responsibility Design Pattern** is a **behavioral pattern** that allows you to pass a request along a chain of handlers. Each handler can either process the request or pass it to the next handler in the chain.

16.2 Problem it Solves

You want to decouple the sender and receiver of a request, and allow multiple potential handlers without hardwiring them into your logic.

16.3 What is Chain of Responsibility?

It's a pattern where a group of objects (handlers) are linked in a chain and the request is passed along until it is handled.

16.4 Real-World Analogy

A customer complaint moves from customer care to supervisor to manager. If one level can't resolve it, it gets escalated to the next.

16.5 Structure

```
package com.mannemlokesesh;  
  
public abstract class Handler {  
    protected Handler next;  
  
    public void setNext(Handler next) {  
        this.next = next;  
    }  
  
    public abstract void handleRequest(String request);  
}
```

16.6 Benefits

- Reduces coupling
- Adds flexibility in assigning responsibilities
- Can add/remove handlers dynamically

16.7 Implementation Steps

1. Create an abstract handler with a next reference
2. Implement concrete handlers
3. Chain the handlers
4. Send requests to the first handler

16.8 Examples

Example1: Logging System

```
package com.mannemlokesesh;
```

```
abstract class Logger {
    public static int INFO = 1;
    public static int DEBUG = 2;
    public static int ERROR = 3;

    protected int level;
    protected Logger nextLogger;

    public void setNextLogger(Logger nextLogger) {
        this.nextLogger = nextLogger;
    }

    public void logMessage(int level, String message) {
        if (this.level <= level) {
            write(message);
        }
        if (nextLogger != null) {
            nextLogger.logMessage(level, message);
        }
    }

    protected abstract void write(String message);
}

class ConsoleLogger extends Logger {
    public ConsoleLogger(int level) {
        this.level = level;
    }

    protected void write(String message) {
        System.out.println("Console: " + message);
    }
}

class FileLogger extends Logger {
    public FileLogger(int level) {
        this.level = level;
    }

    protected void write(String message) {
        System.out.println("File: " + message);
    }
}

class ErrorLogger extends Logger {
    public ErrorLogger(int level) {
        this.level = level;
    }

    protected void write(String message) {
        System.out.println("Error: " + message);
    }
}

public class Example1LoggerDemo {
    private static Logger getChainOfLoggers() {
        Logger errorLogger = new ErrorLogger(Logger.ERROR);
        Logger fileLogger = new FileLogger(Logger.DEBUG);
    }
}
```

```
        Logger consoleLogger = new ConsoleLogger(Logger.INFO);

        errorLogger.setNextLogger(fileLogger);
        fileLogger.setNextLogger(consoleLogger);

        return errorLogger;
    }

    public static void main(String[] args) {
        Logger loggerChain = getChainOfLoggers();
        loggerChain.logMessage(Logger.INFO, "This is an info.");
        loggerChain.logMessage(Logger.DEBUG, "This is a debug.");
        loggerChain.logMessage(Logger.ERROR, "This is an error.");
    }
}
```

Output:

```
Console: This is an info.
File: This is a debug.
Console: This is a debug.
Error: This is an error.
File: This is an error.
Console: This is an error.
```

Example2: Help Desk Support

```
package com.mannemlokes;

abstract class SupportHandler {
    protected SupportHandler next;

    public void setNext(SupportHandler next) {
        this.next = next;
    }

    public abstract void handle(String issue);
}

class LevelOneSupport extends SupportHandler {
    public void handle(String issue) {
        if (issue.contains("password")) {
            System.out.println("Level 1 solved: " + issue);
        } else if (next != null) {
            next.handle(issue);
        }
    }
}

class LevelTwoSupport extends SupportHandler {
    public void handle(String issue) {
        if (issue.contains("network")) {
            System.out.println("Level 2 solved: " + issue);
        } else if (next != null) {
            next.handle(issue);
        }
    }
}
```

```

class ManagerSupport extends SupportHandler {
    public void handle(String issue) {
        System.out.println("Manager resolved: " + issue);
    }
}

public class Example2SupportClient {
    public static void main(String[] args) {
        SupportHandler l1 = new LevelOneSupport();
        SupportHandler l2 = new LevelTwoSupport();
        SupportHandler mgr = new ManagerSupport();

        l1.setNext(l2);
        l2.setNext(mgr);

        l1.handle("password reset");
        l1.handle("network connectivity");
        l1.handle("software crash");
    }
}

```

Output:

```

Level 1 solved: password reset
Level 2 solved: network connectivity
Manager resolved: software crash

```

Example3: Request Processing Pipeline

```

package com.mannemlokes;

abstract class RequestProcessor {
    protected RequestProcessor next;

    public void setNext(RequestProcessor next) {
        this.next = next;
    }

    public void process(String data) {
        String processed = handle(data);
        if (next != null)
            next.process(processed);
    }

    protected abstract String handle(String input);
}

class AuthProcessor extends RequestProcessor {
    protected String handle(String input) {
        System.out.println("Auth Check Passed for: " + input);
        return input;
    }
}

class LoggingProcessor extends RequestProcessor {
    protected String handle(String input) {
        System.out.println("Logging Request: " + input);
    }
}

```

```

        return input;
    }
}

class DataSanitizer extends RequestProcessor {
    protected String handle(String input) {
        String cleaned = input.trim().replaceAll("[^a-zA-Z0-9 ]", "");
        System.out.println("Sanitized Data: " + cleaned);
        return cleaned;
    }
}

public class Example3WebRequestClient {
    public static void main(String[] args) {
        RequestProcessor auth = new AuthProcessor();
        RequestProcessor log = new LoggingProcessor();
        RequestProcessor sanitize = new DataSanitizer();

        auth.setNext(log);
        log.setNext(sanitize);

        auth.process(" User@123! ");
    }
}

```

Output:

```

Auth Check Passed for: User@123!
Logging Request: User@123!
Sanitized Data: User123

```

16.9 Chain vs Other Patterns

Pattern	Purpose
Chain of Responsibility	Pass request through multiple handlers
Command	Encapsulate a request as object
Observer	Notify many listeners of a change

16.10 Pros and Cons

✓ Pros

- Loose coupling
- Dynamic chain modification
- Easy to add new handlers

✗ Cons

- No guarantee request will be handled
- Harder to debug due to delegation

16.11 When to Use / Not to Use

Use When:

- You have multiple handlers for a request
- You want decoupled sender/receiver

Avoid When:

- You want guaranteed single handler
- Logic is better done through a switch-case

16.12 Summary

Feature	Description
Intent	Pass request along a chain of handlers
Use Case	Logging, authorization, processing flows
Key Benefit	Decouples request sender from receiver

The **Chain of Responsibility Pattern** is a flexible way to handle requests dynamically and hierarchically.